Phase 2: TypeScript

타입으로 안전하게 코딩하기 (Day 06~08)

웹 풀스택 PBL 과정

Day 06 - TypeScript 기초: 타입으로 안전하게 코딩하기

TypeScript란 무엇인가?

한 줄 요약

TypeScript는 **JavaScript에 타입(type)이라는 안전장치를 추가한 언어**입니다.

실생활 비유: 도로와 가드레일

JavaScript로 코딩하는 것은 **가드레일 없는 넓은 도로**를 달리는 것과 같습니다. 자유롭지만, 실수하면 바로 낭떠러지로 떨어집니다. TypeScript는 그 도로에 **가드레일을 설치**하는 것입니다. 약간의 제약이 생기지만, 실수해도 큰 사고를 막아줍니다.

비교	JavaScript	TypeScript
자유도	매우 높음	적당히 높음
안전성	낮음 (런타임 에러)	높음 (컴파일 타임 체크)
에러 발견 시점	실행할 때	코드 작성할 때
비유	가드레일 없는 도로	가드레일 있는 도로

왜 TypeScript를 쓰는가? - 실제 에러 사례

JavaScript에서 흔히 발생하는 실수를 봅시다.

```
// JavaScript - 아무 문제 없이 실행되지만, 결과가 이상합니다
function add(a, b) {
  return a + b;
}

console.log(add(10, 20));       // 30 (정상)
console.log(add("10", 20));     // "1020" (문자열 연결이 되어버림!)
console.log(add(10));            // NaN (b가 undefined)
```

위 코드는 JavaScript에서는 에러 없이 실행됩니다. 하지만 결과는 우리가 원하는 것이 아닙니다. TypeScript로 같은 코드를 작성하면 어떻게 될까요?

```
// TypeScript - 잘못된 사용을 미리 잡아줍니다
function add(a: number, b: number): number {
  return a + b;
}

console.log(add(10, 20));       // 30 (정상)
console.log(add("10", 20));     // 컴파일 에러! string은 number가 아닙니다
console.log(add(10));            // 컴파일 에러! 인자가 부족합니다
```

> **핵심 포인트:** TypeScript는 코드를 실행하기 전에 문제를 발견해줍니다. 마치 맞춤법 검사기처럼, 코드를 작성하는 순간 빨간 밑줄로 경고해줍니다.

설치와 실행

```
# 1. TypeScript 설치 (프로젝트 폴더에서)
npm init -y                     # package.json 생성
npm install -D typescript       # TypeScript 설치 (-D는 개발용)

# 2. tsconfig.json 생성 (TypeScript 설정 파일)
npx tsc --init

# 3. ts-node 설치 (TypeScript를 바로 실행할 수 있는 도구)
npm install -D ts-node

# 4. TypeScript 파일 작성
# hello.ts 파일을 만들고 코드를 작성합니다

# 5. 실행 방법 두 가지
npx tsc hello.ts                # hello.js로 변환 후 node hello.js로 실행
npx ts-node hello.ts            # 바로 실행 (개발할 때 편리)
```

> **참고:** `tsc` 는 TypeScript Compiler의 약자입니다. `.ts` 파일을 `.js` 파일로 변환(컴파일)해주는 도구입니다.

기본 타입 어노테이션

"타입 어노테이션"이란 **변수나 함수에 타입 정보를 적어주는 것**입니다. 콜론(`:`) 뒤에 타입을 씁니다.

기본 타입: number, string, boolean

```
// 변수명: 타입 = 값 형태로 작성합니다
let age: number = 25;            // 숫자
let name: string = "김철수";      // 문자열
let isStudent: boolean = true;   // 참/거짓

// 잘못된 값을 넣으면 즉시 에러!
age = "스물다섯";  // 에러! number 타입에 string을 넣을 수 없습니다
name = 42;         // 에러! string 타입에 number를 넣을 수 없습니다
isStudent = 1;     // 에러! boolean 타입에 number를 넣을 수 없습니다
```

배열 타입

```
// 방법 1: 타입[] (더 많이 사용)
let scores: number[] = [90, 85, 92];         // 숫자 배열
let names: string[] = ["김철수", "이영희"];    // 문자열 배열

// 방법 2: Array<타입> (제네릭 문법 - Day 08에서 자세히 배웁니다)
let scores2: Array<number> = [90, 85, 92];
let names2: Array<string> = ["김철수", "이영희"];

// 잘못된 요소를 넣으면 에러!
scores.push("백점");    // 에러! number 배열에 string을 넣을 수 없습니다
```

객체 타입

```
// 객체의 구조(모양)를 타입으로 정의합니다
let student: { name: string; age: number; grade: string } = {
  name: "김철수",
  age: 18,
  grade: "A"
};

// 정의되지 않은 속성에 접근하면 에러!
console.log(student.phone);  // 에러! 'phone'은 정의되지 않은 속성입니다
```

함수 타입

```
// 매개변수와 반환값에 타입을 지정합니다
// (매개변수: 타입, 매개변수: 타입): 반환타입
function multiply(a: number, b: number): number {
  return a * b;
}

// 화살표 함수도 동일합니다
const divide = (a: number, b: number): number => {
  return a / b;
};

// 함수 타입을 변수에 저장할 수도 있습니다
// (a: number, b: number) => number 는 함수의 타입을 설명합니다
let calculator: (a: number, b: number) => number;
calculator = multiply;  // OK
calculator = divide;    // OK
```

void, any, unknown

```typescript
// void: 반환값이 없는 함수에 사용합니다
// 비유: "이 함수는 아무것도 돌려주지 않아요"
function greet(name: string): void {
  console.log(`안녕하세요, ${name}님!`);
  // return 문이 없거나, 값 없이 return만 사용
}

// any: 아무 타입이나 허용 (타입 체크를 포기하는 것)
// 비유: "아무거나 다 넣어도 돼요" - 가능하면 사용하지 마세요!
let anything: any = 42;
anything = "문자열";       // OK (하지만 위험!)
anything = true;           // OK (하지만 위험!)
anything.foo.bar.baz;      // 에러 안 남 (런타임에 터짐!)

// unknown: any보다 안전한 "모르는 타입"
// 비유: "뭔지 모르니까 확인하고 써야 해요"
let mystery: unknown = 42;
mystery = "문자열";        // OK (값 대입은 자유)

// 하지만 사용할 때는 타입 확인이 필수!
// let result: number = mystery + 10;  // 에러! unknown은 바로 사용 불가
if (typeof mystery === "number") {
  let result: number = mystery + 10;   // OK! 타입을 확인했으니 안전
}
```

JS vs TS 비교 코드

```typescript
// === JavaScript 버전 ===
function createUser(name, age, email) {
  return { name, age, email };
}

// 잘못된 호출이지만 JavaScript는 에러를 내지 않습니다
const user1 = createUser("김철수", "스물다섯", 123);
// { name: "김철수", age: "스물다섯", email: 123 } - 이상한 데이터!
```

```
// === TypeScript 버전 ===
function createUser(name: string, age: number, email: string) {
  return { name, age, email };
}

// 잘못된 호출은 즉시 에러!
const user1 = createUser("김철수", "스물다섯", 123);
// 에러! "스물다섯"은 number가 아님, 123은 string이 아님

// 올바른 호출
const user2 = createUser("김철수", 25, "kim@email.com");  // OK!
```

Union Type (유니언 타입)

여러 타입 중 하나를 허용하기

유니언 타입은 `|` (파이프) 기호를 사용해서 **"이 타입 또는 저 타입"**을 표현합니다.

> **실생활 비유:** 동물 카페에서 "이 자리에는 **고양이 또는 강아지**만 앉을 수 있어요"라고 하는 것과 같습니다. 햄스터나 앵무새는 안 됩니다!

```
// id는 숫자 또는 문자열일 수 있습니다
// 예: 1, 2, 3 또는 "abc-123", "def-456"
let id: number | string;

id = 42;          // OK! number
id = "abc-123";   // OK! string
id = true;        // 에러! boolean은 허용되지 않음

// 함수 매개변수에도 사용 가능
function printId(id: number | string): void {
  console.log(`ID: ${id}`);
}

printId(101);         // OK!
printId("abc-202");   // OK!
```

```
// 실용적인 예: API 응답에서 값이 있을 수도, 없을 수도 있을 때
let username: string | null = null;  // 아직 로그인 안 함
username = "김철수";                  // 로그인 후

// 배열에 여러 타입을 넣고 싶을 때
let mixedArray: (number | string)[] = [1, "두", 3, "넷"];
```

Literal Type (리터럴 타입)

정해진 값만 허용하기

리터럴 타입은 **특정 값만 허용**하는 타입입니다.

> **실생활 비유:** 자판기를 생각해보세요. 버튼이 "콜라", "사이다", "주스" 세 개만 있습니다. "라떼"를 누를 수는 없습니다. 정해진 메뉴만 선택 가능합니다!

```
// 옷 사이즈: 정해진 값만 허용
type Size = "small" | "medium" | "large";

let shirtSize: Size = "medium";     // OK!
shirtSize = "large";                // OK!
shirtSize = "extra-large";          // 에러! "extra-large"는 허용되지 않음

// 신호등: 세 가지 색만 가능
type TrafficLight = "red" | "yellow" | "green";

function getAction(light: TrafficLight): string {
  switch (light) {
    case "red":    return "정지";
    case "yellow": return "주의";
    case "green":  return "진행";
  }
}

console.log(getAction("red"));     // "정지"
console.log(getAction("blue"));    // 에러! "blue"는 TrafficLight가 아님
```

```typescript
// 숫자 리터럴도 가능합니다
type DiceNumber = 1 | 2 | 3 | 4 | 5 | 6;

let roll: DiceNumber = 3;    // OK!
roll = 7;                     // 에러! 주사위에 7은 없습니다

// 유니언 타입과 조합하면 강력합니다
type Status = "loading" | "success" | "error";
type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";

function sendRequest(url: string, method: HttpMethod): void {
  console.log(`${method} 요청을 ${url}로 보냅니다`);
}

sendRequest("/api/users", "GET");      // OK!
sendRequest("/api/users", "PATCH");    // 에러! "PATCH"는 HttpMethod가 아님
```

미니 연습문제 1

다음 요구사항에 맞는 타입을 정의해보세요.

```typescript
// 연습: 커피 주문 시스템
// 1. 커피 사이즈는 "tall", "grande", "venti" 중 하나
// 2. 온도는 "hot" 또는 "iced"
// 3. 주문 함수를 만들어보세요

// 여기에 코드를 작성하세요:
type CoffeeSize = ____;
type Temperature = ____;

function orderCoffee(size: CoffeeSize, temp: Temperature): string {
  return `${temp} ${size} 커피 주문 완료!`;
}
```

▶ 정답 보기

Type Narrowing (타입 좁히기)

넓은 타입을 좁혀서 안전하게 사용하기

유니언 타입을 사용하면, 실제로 어떤 타입인지 확인한 후에 사용해야 합니다. 이것을 **타입 좁히기 (Type Narrowing)**라고 합니다.

> **실생활 비유:** 택배 분류 센터를 생각해보세요. 컨베이어 벨트 위에 다양한 크기의 택배가 옵니다. "소형이면 A구역으로, 대형이면 B구역으로" 분류하는 것처럼, 타입도 검사해서 분류합니다.

typeof로 타입 체크

```
// value는 number 또는 string입니다
function formatValue(value: number | string): string {
  // typeof로 타입을 확인합니다
  if (typeof value === "number") {
    // 이 블록 안에서 value는 확실히 number입니다
    return value.toFixed(2);  // 소수점 2자리로 포맷 (number 메서드)
  } else {
    // 이 블록 안에서 value는 확실히 string입니다
    return value.toUpperCase();  // 대문자로 변환 (string 메서드)
  }
}

console.log(formatValue(3.14159));   // "3.14"
console.log(formatValue("hello"));   // "HELLO"
```

여러 타입 좁히기

```
// 세 가지 타입이 가능한 매개변수
function describe(value: number | string | boolean): string {
  if (typeof value === "number") {
    return `숫자: ${value}`;
  } else if (typeof value === "string") {
    return `문자열: "${value}" (${value.length}글자)`;
  } else {
    // 남은 건 boolean밖에 없습니다
    return `불리언: ${value ? "참" : "거짓"}`;
  }
}

console.log(describe(42));         // "숫자: 42"
console.log(describe("안녕"));     // "문자열: "안녕" (2글자)"
console.log(describe(true));       // "불리언: 참"
```

null/undefined 체크

```
// 실무에서 가장 많이 쓰는 패턴!
function getLength(value: string | null): number {
  // null 체크를 먼저 합니다
  if (value === null) {
    return 0;
  }
  // 여기서 value는 확실히 string입니다
  return value.length;
}

console.log(getLength("안녕하세요"));  // 5
console.log(getLength(null));          // 0
```

Discriminated Unions (구별된 유니언)

type 필드로 구별하기

복잡한 객체 유니언에서는 **공통 필드(보통 `type` 또는 `kind`)**를 사용해서 구별합니다.

> **실생활 비유:** 동물원 입장권을 생각해보세요. "성인 입장권"에는 할인 정보가 있고, "아동 입장권"에는 보호자 정보가 있습니다. 입장권의 **종류(type)**를 보고 다르게 처리합니다.

```typescript
// 각 도형을 타입으로 정의합니다
// kind 필드가 각 도형을 구별하는 역할을 합니다
type Circle = {
  kind: "circle";       // 리터럴 타입! "circle"만 가능
  radius: number;        // 반지름
};

type Rectangle = {
  kind: "rectangle";    // 리터럴 타입! "rectangle"만 가능
  width: number;         // 가로
  height: number;        // 세로
};

type Triangle = {
  kind: "triangle";     // 리터럴 타입! "triangle"만 가능
  base: number;          // 밑변
  height: number;        // 높이
};

// Shape는 세 가지 도형 중 하나입니다
type Shape = Circle | Rectangle | Triangle;

// kind로 분기해서 각 도형에 맞는 처리를 합니다
function calculateArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      // 여기서 shape는 Circle 타입으로 좁혀집니다
      return Math.PI * shape.radius ** 2;

    case "rectangle":
      // 여기서 shape는 Rectangle 타입으로 좁혀집니다
      return shape.width * shape.height;

    case "triangle":
      // 여기서 shape는 Triangle 타입으로 좁혀집니다
      return (shape.base * shape.height) / 2;
  }
}

// 사용 예시
const myCircle: Circle = { kind: "circle", radius: 5 };
const myRect: Rectangle = { kind: "rectangle", width: 10, height: 20 };
const myTri: Triangle = { kind: "triangle", base: 6, height: 8 };

console.log(calculateArea(myCircle));  // 78.539... (원의 넓이)
console.log(calculateArea(myRect));    // 200 (직사각형의 넓이)
console.log(calculateArea(myTri));     // 24 (삼각형의 넓이)
```

실전 예제: API 응답 처리

```typescript
// API 응답은 성공 또는 실패 두 가지 형태입니다
type SuccessResponse = {
  status: "success";      // 구별자
  data: string[];         // 성공 시 데이터가 있음
  total: number;
};

type ErrorResponse = {
  status: "error";        // 구별자
  message: string;        // 실패 시 에러 메시지가 있음
  code: number;
};

type ApiResponse = SuccessResponse | ErrorResponse;

function handleResponse(response: ApiResponse): void {
  switch (response.status) {
    case "success":
      // 여기서 response는 SuccessResponse로 좁혀집니다
      console.log(`${response.total}개의 데이터를 받았습니다`);
      response.data.forEach((item) => console.log(`- ${item}`));
      break;

    case "error":
      // 여기서 response는 ErrorResponse로 좁혀집니다
      console.log(`에러 ${response.code}: ${response.message}`);
      break;
  }
}

// 테스트
const success: ApiResponse = {
  status: "success",
  data: ["사과", "바나나", "체리"],
  total: 3,
};

const error: ApiResponse = {
  status: "error",
  message: "서버 연결 실패",
  code: 500,
};

handleResponse(success);  // "3개의 데이터를 받았습니다" + 각 항목 출력
handleResponse(error);    // "에러 500: 서버 연결 실패"
```

자주 하는 실수

실수 1: any 남용

```
// 나쁜 예 - any를 쓰면 TypeScript를 쓰는 의미가 없습니다
function process(data: any): any {
  return data.name.toUpperCase();  // 에러 체크 안 됨!
}

// 좋은 예 - 구체적인 타입을 지정합니다
function process(data: { name: string }): string {
  return data.name.toUpperCase();  // 타입 안전!
}
```

실수 2: 타입 단언(as) 과다 사용

```
// 나쁜 예 - as로 강제 변환 (위험!)
const value: unknown = "hello";
const length = (value as string).length;  // 만약 value가 string이 아니면?

// 좋은 예 - typeof로 확인 후 사용
if (typeof value === "string") {
  const length = value.length;  // 안전!
}
```

실수 3: 객체 속성 오타

```
type User = { name: string; email: string };

// TypeScript가 오타를 잡아줍니다!
const user: User = {
  name: "김철수",
  emial: "kim@test.com",  // 에러! 'emial'은 없는 속성 (email 오타)
};
```

미니 연습문제 2

다음 함수를 완성해보세요.

```typescript
// 연습: 결제 수단별 처리 함수
// 카드 결제: 카드번호 마스킹 처리
// 현금 결제: 거스름돈 계산
// 포인트 결제: 잔여 포인트 계산

type CardPayment = {
  method: "card";
  cardNumber: string;
  amount: number;
};

type CashPayment = {
  method: "cash";
  received: number;    // 받은 금액
  amount: number;      // 결제 금액
};

type PointPayment = {
  method: "point";
  currentPoints: number;  // 현재 보유 포인트
  amount: number;
};

type Payment = CardPayment | CashPayment | PointPayment;

// 이 함수를 완성하세요!
function processPayment(payment: Payment): string {
  // switch-case로 method에 따라 분기해보세요
  // 힌트: 각 case에서 payment의 타입이 자동으로 좁혀집니다
}
```

▶ 정답 보기

Day 07 - 인터페이스와 타입 활용

Interface란?

계약서 비유

인터페이스(Interface)는 **계약서**와 같습니다. "이 객체는 반드시 이런 형태여야 합니다"라고 약속하는 것입니다.

집을 지을 때 설계도(blueprint)가 있어야 하듯이, 객체를 만들 때 인터페이스가 설계도 역할을 합니다.

```typescript
// 인터페이스 정의 = 설계도 작성
interface User {
  name: string;       // 이름은 반드시 문자열
  age: number;        // 나이는 반드시 숫자
  email: string;      // 이메일은 반드시 문자열
}

// 인터페이스에 맞게 객체 생성 = 설계도대로 건축
const user1: User = {
  name: "김철수",
  age: 25,
  email: "kim@test.com",
};

// 설계도와 다르면 에러!
const user2: User = {
  name: "이영희",
  age: 23,
  // email이 빠짐 → 에러! 'email' 속성이 필요합니다
};
```

type vs interface 차이

```typescript
// type으로 정의 (타입 별칭)
type UserType = {
  name: string;
  age: number;
};

// interface로 정의 (인터페이스)
interface UserInterface {
  name: string;
  age: number;
}

// 사용할 때는 동일합니다
const user1: UserType = { name: "김철수", age: 25 };
const user2: UserInterface = { name: "이영희", age: 23 };
```

기능	type	interface
객체 타입 정의	O	O
유니언 타입	O (`string | number`)	X
리터럴 타입	O (`"a" | "b"`)	X
확장(extends)	& 연산자 사용	extends 키워드
선언 병합	X	O (같은 이름으로 추가 가능)
클래스 implements	O	O

> **실무 가이드:** 객체의 형태를 정의할 때는 `interface`, 유니언이나 리터럴 등 복잡한 타입 조합은 `type` 을 사용하는 것이 일반적인 관례입니다.

Interface 정의와 사용

선택적 프로퍼티 (`?`)

모든 속성이 항상 필요한 것은 아닙니다. `?` 를 붙이면 **있어도 되고 없어도 되는 속성**이 됩니다.

```typescript
interface Product {
  name: string;              // 필수
  price: number;             // 필수
  description?: string;      // 선택 (있어도 되고 없어도 됨)
  discount?: number;         // 선택
}

// description과 discount가 없어도 OK!
const apple: Product = {
  name: "사과",
  price: 3000,
};

// 모든 속성을 다 넣어도 OK!
const banana: Product = {
  name: "바나나",
  price: 2000,
  description: "필리핀산 바나나",
  discount: 10,
};
```

읽기 전용 프로퍼티 (`readonly`)

한 번 설정하면 **변경할 수 없는** 속성입니다.

```typescript
interface Config {
  readonly apiUrl: string;        // 읽기 전용! 변경 불가
  readonly maxRetries: number;    // 읽기 전용! 변경 불가
  timeout: number;                // 일반 속성, 변경 가능
}

const config: Config = {
  apiUrl: "https://api.example.com",
  maxRetries: 3,
  timeout: 5000,
};

config.timeout = 10000;      // OK! 일반 속성은 변경 가능
config.apiUrl = "다른 URL";  // 에러! readonly 속성은 변경 불가
config.maxRetries = 5;        // 에러! readonly 속성은 변경 불가
```

메서드 시그니처

인터페이스에 **함수(메서드)**도 정의할 수 있습니다.

```typescript
interface Calculator {
  // 속성
  brand: string;

  // 메서드: 두 가지 표기법
  add(a: number, b: number): number;         // 방법 1: 간결한 표기
  subtract: (a: number, b: number) => number; // 방법 2: 화살표 표기
}

const myCalc: Calculator = {
  brand: "카시오",
  add(a, b) {
    return a + b;
  },
  subtract: (a, b) => {
    return a - b;
  },
};

console.log(myCalc.add(10, 5));        // 15
console.log(myCalc.subtract(10, 5));   // 5
```

---

extends (인터페이스 확장)

기본 회원에서 프리미엄 회원으로

> **실생활 비유:** 기본 회원은 이름과 이메일이 있습니다. 프리미엄 회원은 기본 회원의 모든 정보에 **추가로** 결제 정보와 등급이 있습니다. 처음부터 다시 쓸 필요 없이, 기존 것을 확장합니다.

```typescript
// 기본 회원 인터페이스
interface BasicMember {
  name: string;
  email: string;
  joinDate: string;
}

// 프리미엄 회원 = 기본 회원 + 추가 속성
// extends로 BasicMember의 모든 속성을 물려받습니다
interface PremiumMember extends BasicMember {
  plan: "monthly" | "yearly";    // 추가: 결제 플랜
  cardNumber: string;             // 추가: 카드 번호
  discount: number;               // 추가: 할인율
}

// PremiumMember는 BasicMember의 속성 + 자신의 속성을 모두 가져야 합니다
const premiumUser: PremiumMember = {
  // BasicMember에서 물려받은 속성
  name: "김철수",
  email: "kim@test.com",
  joinDate: "2026-01-01",
  // PremiumMember 고유 속성
  plan: "yearly",
  cardNumber: "1234-5678",
  discount: 20,
};
```

여러 인터페이스를 동시에 확장

```typescript
interface HasName {
  name: string;
}

interface HasAge {
  age: number;
}

interface HasEmail {
  email: string;
}

// 여러 인터페이스를 동시에 확장할 수 있습니다 (쉼표로 구분)
interface Employee extends HasName, HasAge, HasEmail {
  department: string;
  salary: number;
}

const employee: Employee = {
  name: "이영희",
  age: 30,
  email: "lee@company.com",
  department: "개발팀",
  salary: 5000,
};
```

---

implements (클래스에서 인터페이스 구현)

자격증 비유

> **실생활 비유:** 자격증은 "이 기능을 반드시 할 수 있어야 합니다"라는 요구사항 목록입니다. 예를 들어, "운전면허증"이 있으면 반드시 "운전할 수 있어야" 합니다. `implements` 는 클래스가 인터페이스(자격증)의 요구사항을 모두 구현해야 한다는 뜻입니다.

```typescript
// 인터페이스 = 자격 요건 (반드시 구현해야 할 기능 목록)
interface Printable {
  print(): void;
  getPageCount(): number;
}

// implements로 인터페이스를 구현합니다
// "이 클래스는 Printable 자격을 갖추겠다"는 선언
class Report implements Printable {
  constructor(
    private title: string,
    private pages: number
  ) {}

  // Printable 인터페이스에서 요구하는 메서드를 반드시 구현
  print(): void {
    console.log(`보고서 "${this.title}" 인쇄 중...`);
  }

  // 이것도 반드시 구현해야 합니다
  getPageCount(): number {
    return this.pages;
  }
}

class Invoice implements Printable {
  constructor(
    private amount: number
  ) {}

  print(): void {
    console.log(`청구서 (${this.amount}원) 인쇄 중...`);
  }

  getPageCount(): number {
    return 1;  // 청구서는 항상 1페이지
  }
}

// Printable 인터페이스 타입으로 사용 가능
function printDocument(doc: Printable): void {
  console.log(`총 ${doc.getPageCount()}페이지`);
  doc.print();
}

printDocument(new Report("월간 보고서", 15));  // "총 15페이지" → 인쇄
printDocument(new Invoice(50000));              // "총 1페이지" → 인쇄
```

Utility Types (유틸리티 타입)

TypeScript는 기존 타입을 변환해주는 **내장 유틸리티 타입**을 제공합니다. 매우 자주 사용하므로 꼭 익혀두세요!

Partial<T> - 모든 필드를 선택적으로

> **비유:** "수정할 때는 전부 다 안 바꿔도 돼요. 바꾸고 싶은 것만 바꾸세요."

```
interface User {
  name: string;
  age: number;
  email: string;
}

// Partial<User>는 모든 속성이 선택적(?)이 됩니다
// { name?: string; age?: number; email?: string } 과 동일
function updateUser(id: number, updates: Partial<User>): void {
  console.log(`사용자 ${id} 업데이트:`, updates);
}

// 이름만 바꾸고 싶을 때 - 다른 필드는 안 넣어도 OK!
updateUser(1, { name: "새이름" });

// 나이와 이메일만 바꾸고 싶을 때
updateUser(2, { age: 30, email: "new@test.com" });

// 아무것도 안 바꿔도 OK (빈 객체)
updateUser(3, {});
```

Required<T> - 모든 필드를 필수로

> **비유:** "선택이었던 것도 이번에는 전부 다 채워주세요!"

```typescript
interface FormData {
  name?: string;        // 선택적
  email?: string;       // 선택적
  phone?: string;       // 선택적
}

// Required<FormData>는 모든 속성이 필수가 됩니다
// { name: string; email: string; phone: string } 과 동일
function submitForm(data: Required<FormData>): void {
  console.log("폼 제출:", data);
}

// 모든 필드를 다 채워야 합니다
submitForm({
  name: "김철수",
  email: "kim@test.com",
  phone: "010-1234-5678",
});

// 하나라도 빠지면 에러!
// submitForm({ name: "김철수" });  // 에러! email, phone이 없음
```

Pick<T, K> - 특정 필드만 선택

> **비유:** 뷔페에서 "전체 메뉴 중에 이것과 이것만 가져갈게요."

```typescript
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  createdAt: string;
}

// 목록에서 보여줄 정보만 골라냅니다 (비밀번호는 제외!)
type UserListItem = Pick<User, "id" | "name" | "email">;
// 결과: { id: number; name: string; email: string }

const listItem: UserListItem = {
  id: 1,
  name: "김철수",
  email: "kim@test.com",
  // password는 포함 안 됨 - 안전!
};
```

Omit<T, K> - 특정 필드 제외

> **비유:** "전체 메뉴에서 이것만 빼주세요."

```typescript
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  createdAt: string;
}

// 사용자 생성 시에는 id와 createdAt이 서버에서 자동 생성됩니다
// 그래서 입력 타입에서는 제외합니다
type CreateUserInput = Omit<User, "id" | "createdAt">;
// 결과: { name: string; email: string; password: string }

const newUser: CreateUserInput = {
  name: "김철수",
  email: "kim@test.com",
  password: "secure123",
  // id와 createdAt은 서버에서 자동 생성
};
```

Record<K, V> - 키-값 매핑

> **비유:** "이 키로 열면 이런 값이 나옵니다"라는 사물함 시스템

```typescript
// 과목별 점수를 저장하는 타입
type Subject = "korean" | "english" | "math";
type ScoreBoard = Record<Subject, number>;
// 결과: { korean: number; english: number; math: number }

const scores: ScoreBoard = {
  korean: 90,
  english: 85,
  math: 95,
};

// 상태별 메시지 매핑
type Status = "loading" | "success" | "error";
const statusMessages: Record<Status, string> = {
  loading: "로딩 중입니다...",
  success: "성공적으로 완료되었습니다!",
  error: "오류가 발생했습니다.",
};

console.log(statusMessages["success"]);  // "성공적으로 완료되었습니다!"
```

타입 조합 패턴

Intersection Type (`&`) - 교차 타입

유니언(`|`)이 "이것 **또는** 저것"이라면, 교차(`&`)는 "이것 **그리고** 저것"입니다.

```typescript
type HasName = {
  name: string;
};

type HasAge = {
  age: number;
};

type HasEmail = {
  email: string;
};

// & 로 합치면 모든 속성을 가져야 합니다
type Person = HasName & HasAge & HasEmail;
// 결과: { name: string; age: number; email: string }

const person: Person = {
  name: "김철수",
  age: 25,
  email: "kim@test.com",
};
```

실전: 타임스탬프 추가 패턴

```typescript
// 공통으로 추가할 타임스탬프 타입
type WithTimestamp = {
  createdAt: string;
  updatedAt: string;
};

// 공통으로 추가할 ID 타입
type WithId = {
  id: number;
};

// 기본 사용자 데이터
type UserData = {
  name: string;
  email: string;
};

// 기본 게시글 데이터
type PostData = {
  title: string;
  content: string;
};

// 조합! DB에 저장된 사용자 = ID + 사용자데이터 + 타임스탬프
type UserRecord = WithId & UserData & WithTimestamp;

// 조합! DB에 저장된 게시글 = ID + 게시글데이터 + 타임스탬프
type PostRecord = WithId & PostData & WithTimestamp;

const user: UserRecord = {
  id: 1,
  name: "김철수",
  email: "kim@test.com",
  createdAt: "2026-01-01",
  updatedAt: "2026-03-05",
};
```

실습: Todo 앱 타입 설계

지금까지 배운 것을 활용해서 **Todo 앱의 타입 시스템**을 설계해봅시다.

```typescript
// ===== Todo 앱 타입 설계 =====

// 1. 우선순위 타입 (리터럴 타입)
type Priority = "low" | "medium" | "high";

// 2. Todo 상태 타입 (리터럴 타입)
type TodoStatus = "todo" | "in-progress" | "done";

// 3. Todo 인터페이스 (메인 데이터 구조)
interface Todo {
  readonly id: number;          // 읽기 전용 ID
  title: string;                // 할 일 제목
  description?: string;         // 설명 (선택적)
  priority: Priority;           // 우선순위
  status: TodoStatus;           // 상태
  dueDate?: string;             // 마감일 (선택적)
  tags: string[];               // 태그 목록
  createdAt: string;            // 생성일
  updatedAt: string;            // 수정일
}

// 4. Todo 생성 시 입력 타입 (id, createdAt, updatedAt은 자동 생성)
type CreateTodoInput = Omit<Todo, "id" | "createdAt" | "updatedAt">;

// 5. Todo 수정 시 입력 타입 (모든 필드가 선택적)
type UpdateTodoInput = Partial<Omit<Todo, "id" | "createdAt" | "updatedAt">>;

// 6. Todo 목록 표시용 간략 타입
type TodoSummary = Pick<Todo, "id" | "title" | "priority" | "status">;

// 7. 필터 조건 인터페이스
interface TodoFilter {
  status?: TodoStatus;
  priority?: Priority;
  search?: string;
}

// ===== 사용 예시 =====

// Todo 생성
const newTodo: CreateTodoInput = {
  title: "TypeScript 공부하기",
  description: "Day 06~08 교안 정리",
  priority: "high",
  status: "in-progress",
  tags: ["study", "typescript"],
};

// Todo 수정 (제목만 변경)
const updates: UpdateTodoInput = {
  title: "TypeScript 마스터하기",
};
```

```typescript
// Todo 목록 표시
const todoList: TodoSummary[] = [
  { id: 1, title: "TypeScript 공부하기", priority: "high", status: "in-progress" },
  { id: 2, title: "React 프로젝트 시작", priority: "medium", status: "todo" },
];

// 필터 적용
const filter: TodoFilter = {
  status: "todo",
  priority: "high",
};

// 필터링 함수
function filterTodos(todos: Todo[], filter: TodoFilter): Todo[] {
  return todos.filter((todo) => {
    if (filter.status && todo.status !== filter.status) return false;
    if (filter.priority && todo.priority !== filter.priority) return false;
    if (filter.search && !todo.title.includes(filter.search)) return false;
    return true;
  });
}
```

미니 연습문제 3

위의 Todo 앱 타입을 참고하여, **쇼핑몰 상품(Product)** 타입 시스템을 설계해보세요.

```typescript
// 연습: 쇼핑몰 상품 타입 설계
// 요구사항:
// 1. 카테고리: "electronics" | "clothing" | "food"
// 2. 상품 인터페이스: id, name, price, category, description?, stock, imageUrl?
// 3. 상품 생성 입력 타입 (id 제외)
// 4. 상품 수정 입력 타입 (모든 필드 선택적, id 제외)
// 5. 상품 목록용 간략 타입 (id, name, price, category만)

// 여기에 코드를 작성하세요!
```

▶ 정답 보기

Day 08 - 제네릭: 재사용 가능한 타입

제네릭이란?

만능 상자 비유

> **실생활 비유:** 선물 상자를 생각해보세요. 상자의 크기와 모양은 같지만, 안에 넣는 물건은 다릅니다. 어떤 상자에는 책을, 어떤 상자에는 장난감을, 어떤 상자에는 옷을 넣습니다. **상자의 형태는 같되, 내용물의 종류만 달라지는 것** - 이것이 바로 제네릭입니다.

왜 any 대신 제네릭인가?

```typescript
// 방법 1: any 사용 (나쁜 방법)
function getFirstAny(arr: any[]): any {
  return arr[0];
}

const firstNum = getFirstAny([1, 2, 3]);
// firstNum의 타입은 any → 타입 정보가 사라짐!
// firstNum.toFixed(2)를 써도 자동완성이 안 됨

// 방법 2: 타입별로 함수를 따로 만들기 (비효율적)
function getFirstNumber(arr: number[]): number {
  return arr[0];
}
function getFirstString(arr: string[]): string {
  return arr[0];
}
// 타입이 추가될 때마다 함수를 새로 만들어야 함... 비효율적!

// 방법 3: 제네릭 사용 (최선의 방법!)
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

const num = getFirst<number>([1, 2, 3]);      // num의 타입: number
const str = getFirst<string>(["a", "b"]);      // str의 타입: string
const bool = getFirst([true, false]);           // bool의 타입: boolean (자동 추론!)

// 타입 정보가 유지되므로 자동완성이 잘 됩니다!
num.toFixed(2);         // OK! number 메서드 자동완성
str.toUpperCase();      // OK! string 메서드 자동완성
```

`<T>` - 타입 변수

`T`는 **타입을 담는 변수**입니다. 일반 변수가 값을 담듯이, `T`는 타입을 담습니다.

> **비유:** 자판기의 음료 종류 슬롯과 같습니다. 자판기의 구조는 동일하지만, 어떤 슬롯에는 콜라를, 어떤 슬롯에는 사이다를 넣습니다. `T`는 "여기에 어떤 타입이든 들어올 수 있다"는 빈 슬롯입니다.

```typescript
// T는 관례적인 이름입니다. 아무 이름이나 사용 가능합니다.
// T = Type의 약자
// E = Element (배열 요소에 자주 사용)
// K = Key, V = Value (키-값 쌍에 사용)

function identity<T>(value: T): T {
  return value;
}

// T에 number가 들어가면 → function identity(value: number): number
identity<number>(42);

// T에 string이 들어가면 → function identity(value: string): string
identity<string>("hello");

// TypeScript가 자동으로 추론해줍니다 (타입 생략 가능)
identity(42);         // T는 number로 자동 추론
identity("hello");    // T는 string으로 자동 추론
```

제네릭 함수

배열 유틸리티 함수들

```typescript
// 배열의 첫 번째 요소를 반환하는 함수
function getFirst<T>(arr: T[]): T | undefined {
  // 빈 배열일 수 있으므로 undefined도 반환 가능
  return arr[0];
}

console.log(getFirst([10, 20, 30]));       // 10 (number)
console.log(getFirst(["사과", "바나나"])); // "사과" (string)
console.log(getFirst([]));                  // undefined

// 배열의 마지막 요소를 반환하는 함수
function getLast<T>(arr: T[]): T | undefined {
  return arr[arr.length - 1];
}

console.log(getLast([10, 20, 30]));  // 30

// 조건에 맞는 요소만 필터링하는 함수
function filterBy<T>(arr: T[], predicate: (item: T) => boolean): T[] {
  // predicate는 "각 요소를 받아서 true/false를 반환하는 함수"입니다
  const result: T[] = [];
  for (const item of arr) {
    if (predicate(item)) {
      result.push(item);
    }
  }
  return result;
}

// 짝수만 필터링
const evenNumbers = filterBy([1, 2, 3, 4, 5, 6], (n) => n % 2 === 0);
console.log(evenNumbers);  // [2, 4, 6]

// 5글자 이상인 문자열만 필터링
const longWords = filterBy(
  ["apple", "hi", "banana", "ok", "cherry"],
  (word) => word.length >= 5
);
console.log(longWords);  // ["apple", "banana", "cherry"]
```

두 개의 타입 변수

```typescript
// K(Key)와 V(Value) 두 개의 타입 변수 사용
function createPair<K, V>(key: K, value: V): { key: K; value: V } {
  return { key, value };
}

const pair1 = createPair("name", "김철수");
// pair1의 타입: { key: string; value: string }

const pair2 = createPair(1, true);
// pair2의 타입: { key: number; value: boolean }

const pair3 = createPair("age", 25);
// pair3의 타입: { key: string; value: number }
```

```typescript
// 두 배열을 합쳐서 쌍(pair)으로 만드는 함수
function zip<A, B>(arrA: A[], arrB: B[]): [A, B][] {
  // 더 짧은 배열의 길이에 맞춥니다
  const length = Math.min(arrA.length, arrB.length);
  const result: [A, B][] = [];

  for (let i = 0; i < length; i++) {
    result.push([arrA[i], arrB[i]]);
  }
  return result;
}

const names = ["김철수", "이영희", "박민수"];
const scores = [90, 85, 95];

const nameScorePairs = zip(names, scores);
// 결과: [["김철수", 90], ["이영희", 85], ["박민수", 95]]
// 타입: [string, number][]
```

제네릭 인터페이스

ApiResponse<T> - API 응답 공통 타입

실무에서 가장 많이 사용하는 패턴 중 하나입니다.

```typescript
// API 응답의 공통 구조를 제네릭으로 정의
interface ApiResponse<T> {
  success: boolean;      // 성공 여부
  data: T;               // 실제 데이터 (타입이 매번 다름!)
  message: string;       // 응답 메시지
  timestamp: string;     // 응답 시간
}

// 사용자 데이터를 담을 때
interface User {
  id: number;
  name: string;
  email: string;
}

// T에 User가 들어감 → data의 타입이 User가 됨
const userResponse: ApiResponse<User> = {
  success: true,
  data: {
    id: 1,
    name: "김철수",
    email: "kim@test.com",
  },
  message: "사용자 조회 성공",
  timestamp: "2026-03-05T10:00:00",
};

// 상품 목록을 담을 때
interface Product {
  id: number;
  name: string;
  price: number;
}

// T에 Product[]가 들어감 → data의 타입이 Product[]가 됨
const productListResponse: ApiResponse<Product[]> = {
  success: true,
  data: [
    { id: 1, name: "노트북", price: 1500000 },
    { id: 2, name: "마우스", price: 50000 },
  ],
  message: "상품 목록 조회 성공",
  timestamp: "2026-03-05T10:05:00",
};

// 페이지네이션 응답
interface PaginatedData<T> {
  items: T[];            // 데이터 목록
  total: number;         // 전체 개수
  page: number;          // 현재 페이지
  pageSize: number;      // 페이지당 개수
}
```

```typescript
// 이중 제네릭! ApiResponse 안에 PaginatedData 안에 User
const paginatedUsers: ApiResponse<PaginatedData<User>> = {
  success: true,
  data: {
    items: [
      { id: 1, name: "김철수", email: "kim@test.com" },
      { id: 2, name: "이영희", email: "lee@test.com" },
    ],
    total: 50,
    page: 1,
    pageSize: 10,
  },
  message: "사용자 목록 조회 성공",
  timestamp: "2026-03-05T10:10:00",
};
```

DataStore<T> - 범용 데이터 저장소

```typescript
// 어떤 타입의 데이터든 저장하고 관리하는 저장소
interface DataStore<T> {
  getAll(): T[];                          // 전체 조회
  getById(id: number): T | undefined;     // ID로 조회
  add(item: T): void;                     // 추가
  update(id: number, item: Partial<T>): void;  // 수정
  remove(id: number): void;               // 삭제
}

// 사용자 저장소로 사용
interface User {
  id: number;
  name: string;
  email: string;
}

// DataStore<User>를 구현하는 클래스
class UserStore implements DataStore<User> {
  private users: User[] = [];  // 내부 저장소

  getAll(): User[] {
    return [...this.users];  // 복사본 반환 (원본 보호)
  }

  getById(id: number): User | undefined {
    return this.users.find((u) => u.id === id);
  }

  add(user: User): void {
    this.users.push(user);
  }

  update(id: number, updates: Partial<User>): void {
    const index = this.users.findIndex((u) => u.id === id);
    if (index !== -1) {
      this.users[index] = { ...this.users[index], ...updates };
    }
  }

  remove(id: number): void {
    this.users = this.users.filter((u) => u.id !== id);
  }
}

// 사용 예시
const store = new UserStore();
store.add({ id: 1, name: "김철수", email: "kim@test.com" });
store.add({ id: 2, name: "이영희", email: "lee@test.com" });

console.log(store.getAll());
// [{ id: 1, name: "김철수", ... }, { id: 2, name: "이영희", ... }]
```

```
store.update(1, { name: "김철수(수정)" });
console.log(store.getById(1));
// { id: 1, name: "김철수(수정)", email: "kim@test.com" }
```

제네릭 제약 조건 (Constraints)

extends로 제한하기

제네릭은 아무 타입이나 받을 수 있지만, 때로는 **최소한의 조건**을 만족하는 타입만 받고 싶을 때가 있습니다.

> **실생활 비유:** "이 놀이기구는 키 120cm 이상만 탈 수 있습니다." 아무나 다 타는 게 아니라, **최소 조건**을 만족해야 합니다. 제네릭 제약 조건도 마찬가지입니다.

```typescript
// T는 아무 타입이나 가능 → 하지만 id가 없는 타입이 들어오면?
function findById_unsafe<T>(items: T[], id: number): T | undefined {
  // 에러! T에 id 속성이 있다는 보장이 없음
  // return items.find(item => item.id === id);
  return undefined;
}

// T extends { id: number } → "T는 최소한 id: number를 가져야 함"
function findById<T extends { id: number }>(
  items: T[],
  id: number
): T | undefined {
  // OK! T에 id 속성이 반드시 있으므로 안전하게 접근 가능
  return items.find((item) => item.id === id);
}

// id가 있는 타입은 OK!
interface User {
  id: number;
  name: string;
}

interface Product {
  id: number;
  name: string;
  price: number;
}

const users: User[] = [
  { id: 1, name: "김철수" },
  { id: 2, name: "이영희" },
];

const products: Product[] = [
  { id: 1, name: "노트북", price: 1500000 },
  { id: 2, name: "마우스", price: 50000 },
];

findById(users, 1);      // OK! User에 id가 있음
findById(products, 2);   // OK! Product에 id가 있음

// id가 없는 타입은 에러!
const strings = ["a", "b", "c"];
// findById(strings, 1);  // 에러! string에는 id 속성이 없음
```

keyof와 함께 사용

`keyof` 는 **객체 타입의 키(속성 이름)를 유니언 타입으로 추출**합니다.

```typescript
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
}

// keyof User = "id" | "name" | "email" | "age"
type UserKeys = keyof User;

// 객체에서 특정 속성의 값을 안전하게 가져오는 함수
// K extends keyof T → K는 T의 속성 이름 중 하나여야 함
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user: User = {
  id: 1,
  name: "김철수",
  email: "kim@test.com",
  age: 25,
};

const name = getProperty(user, "name");      // 타입: string
const age = getProperty(user, "age");        // 타입: number
// getProperty(user, "phone");               // 에러! "phone"은 User의 키가 아님
```

```typescript
// 실전 예제: 객체의 특정 속성들만 뽑아내는 함수
function pickProperties<T, K extends keyof T>(
  obj: T,
  keys: K[]
): Pick<T, K> {
  const result = {} as Pick<T, K>;
  for (const key of keys) {
    result[key] = obj[key];
  }
  return result;
}

const user = { id: 1, name: "김철수", email: "kim@test.com", age: 25 };

// name과 email만 뽑아내기
const summary = pickProperties(user, ["name", "email"]);
// 결과: { name: "김철수", email: "kim@test.com" }
// 타입: Pick<User, "name" | "email">
```

제네릭 실전 패턴

Stack (스택) - 후입선출 자료구조

```
// 스택: 마지막에 넣은 것이 먼저 나옴 (접시 쌓기와 같음)
// T를 사용해 어떤 타입의 스택이든 만들 수 있습니다
class Stack<T> {
  private items: T[] = [];  // 내부 저장소

  // 맨 위에 추가
  push(item: T): void {
    this.items.push(item);
  }

  // 맨 위에서 꺼내기
  pop(): T | undefined {
    return this.items.pop();
  }

  // 맨 위 요소 확인 (꺼내지 않음)
  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  // 스택이 비었는지 확인
  isEmpty(): boolean {
    return this.items.length === 0;
  }

  // 스택의 크기
  get size(): number {
    return this.items.length;
  }
}

// 숫자 스택
const numberStack = new Stack<number>();
numberStack.push(10);
numberStack.push(20);
numberStack.push(30);
console.log(numberStack.pop());   // 30 (마지막에 넣은 것이 먼저 나옴)
console.log(numberStack.peek());  // 20 (현재 맨 위)
console.log(numberStack.size);    // 2

// 문자열 스택 (같은 클래스를 재사용!)
const stringStack = new Stack<string>();
stringStack.push("첫 번째");
stringStack.push("두 번째");
console.log(stringStack.pop());  // "두 번째"
```

Queue (큐) - 선입선출 자료구조

```typescript
// 큐: 먼저 넣은 것이 먼저 나옴 (줄 서기와 같음)
class Queue<T> {
  private items: T[] = [];

  // 뒤에 추가 (줄의 맨 뒤에 서기)
  enqueue(item: T): void {
    this.items.push(item);
  }

  // 앞에서 꺼내기 (줄의 맨 앞 사람이 나가기)
  dequeue(): T | undefined {
    return this.items.shift();
  }

  // 맨 앞 요소 확인
  front(): T | undefined {
    return this.items[0];
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }

  get size(): number {
    return this.items.length;
  }
}

// 주문 처리 시스템
interface Order {
  id: number;
  menu: string;
  customer: string;
}

const orderQueue = new Queue<Order>();

// 주문 접수 (뒤에 추가)
orderQueue.enqueue({ id: 1, menu: "아메리카노", customer: "김철수" });
orderQueue.enqueue({ id: 2, menu: "라떼", customer: "이영희" });
orderQueue.enqueue({ id: 3, menu: "카푸치노", customer: "박민수" });

// 주문 처리 (앞에서부터 처리)
while (!orderQueue.isEmpty()) {
  const order = orderQueue.dequeue()!;
  console.log(`주문 #${order.id}: ${order.customer}님의 ${order.menu} 제조 중...`);
}
// 주문 #1: 김철수님의 아메리카노 제조 중...
// 주문 #2: 이영희님의 라떼 제조 중...
// 주문 #3: 박민수님의 카푸치노 제조 중...
```

이벤트 시스템 (TypedEventEmitter)

```typescript
// 이벤트 이름과 데이터 타입을 매핑하는 제네릭 이벤트 시스템
class TypedEventEmitter<EventMap extends Record<string, any>> {
  // 이벤트별 리스너 목록을 저장
  private listeners: {
    [K in keyof EventMap]?: ((data: EventMap[K]) => void)[];
  } = {};

  // 이벤트 리스너 등록
  on<K extends keyof EventMap>(
    event: K,
    listener: (data: EventMap[K]) => void
  ): void {
    if (!this.listeners[event]) {
      this.listeners[event] = [];
    }
    this.listeners[event]!.push(listener);
  }

  // 이벤트 발생
  emit<K extends keyof EventMap>(event: K, data: EventMap[K]): void {
    const eventListeners = this.listeners[event];
    if (eventListeners) {
      eventListeners.forEach((listener) => listener(data));
    }
  }
}

// 이벤트 타입 정의: 이벤트 이름 → 데이터 타입
interface AppEvents {
  login: { userId: number; username: string };
  logout: { userId: number };
  purchase: { productId: number; amount: number; userId: number };
}

// 타입이 안전한 이벤트 시스템!
const emitter = new TypedEventEmitter<AppEvents>();

// login 이벤트의 데이터 타입이 자동으로 { userId: number; username: string }
emitter.on("login", (data) => {
  console.log(`${data.username}님이 로그인했습니다 (ID: ${data.userId})`);
});

// purchase 이벤트의 데이터 타입이 자동으로 추론됨
emitter.on("purchase", (data) => {
  console.log(`사용자 ${data.userId}가 상품 ${data.productId}를 ${data.amount}원에 구매
});

// 이벤트 발생 시에도 타입 체크!
emitter.emit("login", { userId: 1, username: "김철수" });  // OK!
// emitter.emit("login", { userId: 1 });  // 에러! username이 없음
// emitter.emit("unknown", {});           // 에러! "unknown" 이벤트는 없음
```

파이프라인 패턴

```
// 여러 변환 함수를 연결하는 파이프라인
// 각 함수의 출력이 다음 함수의 입력이 됩니다
class Pipeline<Input, Output> {
  // transform: Input을 Output으로 변환하는 함수
  constructor(private transform: (input: Input) => Output) {}

  // 새로운 변환 단계를 추가 (체이닝)
  // 현재 Output이 다음 단계의 Input이 됩니다
  pipe<NextOutput>(
    nextTransform: (input: Output) => NextOutput
  ): Pipeline<Input, NextOutput> {
    return new Pipeline((input: Input) => {
      const intermediate = this.transform(input);  // 현재 변환 수행
      return nextTransform(intermediate);           // 다음 변환 수행
    });
  }

  // 파이프라인 실행
  execute(input: Input): Output {
    return this.transform(input);
  }
}

// 사용 예시: 문자열 처리 파이프라인
const textProcessor = new Pipeline((input: string) => input.trim())
  .pipe((text) => text.toLowerCase())           // 소문자로 변환
  .pipe((text) => text.replace(/\s+/g, "-"))    // 공백을 하이픈으로
  .pipe((text) => text.slice(0, 50));           // 최대 50글자

const result = textProcessor.execute("  Hello World TypeScript  ");
console.log(result);  // "hello-world-typescript"

// 숫자 처리 파이프라인
const numberProcessor = new Pipeline((input: number) => input * 2)
  .pipe((n) => n + 10)                          // 10 더하기
  .pipe((n) => Math.round(n))                   // 반올림
  .pipe((n) => `결과: ${n}점`);                 // 문자열로 변환

console.log(numberProcessor.execute(42.7));  // "결과: 95점"
// 42.7 → 85.4 → 95.4 → 95 → "결과: 95점"
```

자주 하는 실수와 팁

실수 1: 불필요한 제네릭

```typescript
// 나쁜 예: T를 한 번만 사용하면 제네릭이 불필요합니다
function printValue<T>(value: T): void {
  console.log(value);
}

// 좋은 예: 제네릭 없이 간단하게
function printValue(value: unknown): void {
  console.log(value);
}

// 제네릭이 필요한 경우: T가 여러 곳에서 사용될 때
function firstAndLast<T>(arr: T[]): [T, T] {
  return [arr[0], arr[arr.length - 1]];
  // T가 매개변수(T[])와 반환값([T, T]) 모두에 사용됨 → 제네릭 필요!
}
```

실수 2: 제약 조건 없이 속성 접근

```typescript
// 나쁜 예: T에 length가 있다는 보장이 없음
function logLength<T>(value: T): void {
  // console.log(value.length);  // 에러! T에 length가 있을 수도, 없을 수도
}

// 좋은 예: 제약 조건으로 length 보장
function logLength<T extends { length: number }>(value: T): void {
  console.log(value.length);  // OK! length가 반드시 있음
}

logLength("hello");       // 5 (string에 length 있음)
logLength([1, 2, 3]);     // 3 (배열에 length 있음)
// logLength(42);          // 에러! number에는 length가 없음
```

실수 3: 기본 타입 매개변수를 모르는 것

```typescript
// 제네릭에 기본값을 설정할 수 있습니다 (= 뒤에 기본 타입)
interface Container<T = string> {
  value: T;
}

// 타입을 지정하지 않으면 기본값(string) 사용
const box1: Container = { value: "hello" };           // T = string (기본값)
const box2: Container<number> = { value: 42 };        // T = number (직접 지정)
const box3: Container<boolean> = { value: true };     // T = boolean (직접 지정)
```

팁: 제네릭 네이밍 관례

```
// 관례적인 타입 변수 이름들
// T: Type (가장 일반적)
function identity<T>(value: T): T { return value; }

// K, V: Key, Value (키-값 쌍)
function setProperty<K extends string, V>(key: K, value: V) {}

// E: Element (컬렉션 요소)
class List<E> { /* ... */ }

// R: Return type (반환 타입)
type AsyncFn<R> = () => Promise<R>;

// 여러 타입 변수가 필요하면 T, U, V 순서로 사용
function merge<T, U>(obj1: T, obj2: U): T & U {
  return { ...obj1, ...obj2 };
}
```

미니 연습문제 4

다음 요구사항에 맞는 제네릭 함수를 작성해보세요.

```
// 연습 1: 배열에서 중복을 제거하는 제네릭 함수
// 힌트: Set을 사용하세요
function unique<T>(arr: T[]): T[] {
  // 여기에 코드를 작성하세요
}

// 연습 2: 두 객체를 병합하는 제네릭 함수
// 힌트: 스프레드 연산자를 사용하세요
function merge<T, U>(obj1: T, obj2: U): T & U {
  // 여기에 코드를 작성하세요
}

// 연습 3: 배열을 그룹으로 나누는 제네릭 함수
// groupBy([1,2,3,4,5], n => n % 2 === 0 ? "짝수" : "홀수")
// → { "홀수": [1,3,5], "짝수": [2,4] }
function groupBy<T>(
  arr: T[],
  keyFn: (item: T) => string
): Record<string, T[]> {
  // 여기에 코드를 작성하세요
}
```

▶ 정답 보기

Day 06~08 핵심 요약

주제	핵심 내용
타입 어노테이션	변수, 함수에 `: 타입` 으로 타입 지정
유니언 타입	`A | B` - 여러 타입 중 하나 허용
리터럴 타입	`"a" | "b"` - 정해진 값만 허용
타입 좁히기	`typeof`, `if/else` 로 타입 분기
구별된 유니언	`kind` 같은 공통 필드로 타입 구별
인터페이스	객체의 형태를 정의하는 계약서
extends	인터페이스 확장 (상속)
implements	클래스가 인터페이스를 구현
유틸리티 타입	`Partial`, `Pick`, `Omit`, `Record` 등
교차 타입	`A & B` - 두 타입을 합침
제네릭	`<T>` 로 타입을 매개변수화하여 재사용
제네릭 제약	`<T extends 조건>` 으로 타입 제한
keyof	객체 타입의 키를 유니언으로 추출

> **다음 단계:** Phase 3에서는 React를 배웁니다. TypeScript와 React를 함께 사용하면 컴포넌트의 props, state에도 타입을 지정할 수 있어서 훨씬 안전한 UI 개발이 가능합니다. 오늘 배운 인터페이스와 제네릭이 React에서 매일 사용되므로 잘 복습해두세요!